

Adora — Forward-Only Synthesis (torch-ready)

Design note: a derivative-free 1.5D Stokes forward · Ivan Milic · 2026-09-01 · proposal

Goal. A sibling of `synth3d.py`'s cube synth, guaranteed *forward-only*: it never builds an autodiff tape and never materialises response functions — cheap in memory, safe to embed inside a *torch*-driven inversion where the gradient is supplied elsewhere.

What “saving the derivatives” actually means here

Two distinct costs, both absent from a pure forward: **(1) the Jacobian** — the response path (`jax.jacrev` in `vector_response_fn.py`) materialises $\partial \mathbf{I} / \partial \{\text{params}\}$ of shape $(4, n_\lambda, 8, n_z)$ *per column*, the literal “all the derivatives”; and **(2) the reverse-mode tape** — differentiating through the DELO loop (`fori_loop` in `vector_formal_solver.py`) retains the full n_z -deep Stokes trajectory plus per-layer matrices per column, whereas forward-only runs that loop in $O(1)$ depth memory.

Key fact: `synth_cube` is already a plain `@jax.jit forward` — a jitted program that is never differentiated builds *no* tape (XLA frees intermediates); memory only balloons when something wraps it in `jacrev/grad`. So this is a small, explicit variant — not a rewrite — that makes non-differentiability a hard guarantee, adds a torch-friendly `dtype`, and fixes a clean seam for the torch backward.

Proposed edits to `synth3d.py`

Factor the current `vmap` body into a private `_synth_cube`; keep `synth_cube` exactly as today (differentiable-ready). Add one lean wrapper:

```
from functools import partial
import jax.numpy as jnp
```

```
@partial(jax.jit, static_argnames=("dtype",))          # dtype must be static
def synth_cube_fwd(adata, waves, dz, T, ne, nH, vz, vturb, b, gb, cb,
                    dtype=jnp.float64):                # jnp.float32 -> torch parity, 1/2 mem
    a = [x.astype(dtype) for x in (waves, dz, T, ne, nH, vz, vturb, b, gb, cb)]
    out = _synth_cube(adata, *a)                       # same physics as synth_cube
    return jax.lax.stop_gradient(out)                  # <- constant to any outer autodiff: no t
```

- `stop_gradient` makes the `synth` a *constant* to any enclosing `grad/torch` graph — nothing is stored for a backward pass, and accidental nesting can never build a tape.
- `dtype` (`static` $\Rightarrow$ no recompile churn) gives a `float32` path that halves working memory and matches a torch pipeline; keep `float64` as default for reference accuracy.
- **The process-parallel path is already this.** `synth3d_mp.synth_cube_mp` only runs the forward, blocks, and returns numpy — no tape ever. Point its worker at `synth_cube_fwd` to also get the `stop_gradient/f32` guarantees; no other change.

The one real decision: how does *torch* get the gradient?

Option	What it is	Derivative memory
A. Black-box (<i>recommended first</i>)	JAX forward as a constant; torch wraps it <code>no_grad</code> . Use for training targets, validation, forward spectra.	None. Ships now.
B. Custom-VJP bridge	Expose <code>jax.vjp</code> ; a <code>torch.autograd.Function</code> calls the JAX forward (<code>dlpack</code>) and, in backward, one on-demand $v \cdot J$.	One cotangent $(4, n_\lambda)$ — <i>not</i> the full $(4, n_\lambda, 8, n_z)$ Jacobian.
C. Torch reimpl.	Re-code DELO in torch; <code>torch.utils.checkpoint</code> the depth loop (recompute in backward). JAX forward is the numerical reference.	$O(\sqrt{n_z})$ instead of $O(n_z)$.

Recommendation. Build **A** now (the wrapper above — tiny, safe, immediately useful for generating spectra without OOM). Keep `_synth_cube` pure so **B** slots in later without touching the physics; **C** is the endgame only if the whole inversion moves into torch, where this `stop_gradient` forward doubles as the validation oracle.

Scope. No change to `synth_cube` or the response path — the differentiable stack stays intact; this note proposes only the JAX forward + the interop seam, not a torch reimplementation. If gradients-through-physics *are* wanted later without the full Jacobian, the levers are B's on-demand VJP or `jax.checkpoint` on the DELO loop.